Dominando Métodos de Array no JavaScript

Do `push` ao `reduce` — aprenda a manipular listas de dados como um programador profissional. Nesta apresentação, vamos explorar os métodos mais poderosos dos arrays usando exemplos práticos e diretos ao ponto.

LÓGICA DE PROGRAMAÇÃO · 1º SEMESTRE

O Que São Métodos de Array?

❌ Sem métodos (loop manual)

```
let nomes = [];
for (let i = 0; i < produtos.length; i++) {
  nomes.push(produtos[i].nome);
}
```

Funciona, mas é verboso, repetitivo e fácil de errar.

✅ Com métodos de array

```
let nomes = produtos.map(produto => produto.nome);
```

Uma linha, clara e expressiva. Métodos de array são **funções embutidas** que realizam operações comuns em listas — adicionar, remover, transformar, filtrar e resumir dados — de forma elegante e legível.

Nosso Cenário de Exemplo

Durante toda a apresentação, usaremos **um único array** chamado `produtos`. Pense nele como a prateleira de um mercadinho — três itens, cada um com suas próprias informações.

```
let produtos = [
  { id: 1, nome: "Coca-cola",  preco: 10, estoque: 50 },
  { id: 2, nome: "Chocolate",  preco: 5,  estoque: 20 },
  { id: 3, nome: "Sabonete",   preco: 3,  estoque: 10 }
];
```

Cada objeto representa um produto com: **id** (identificador), **nome**, **preco** e **estoque** (quantidade disponível). Este array será a base de todos os exemplos a seguir.

BLOCO 1

Métodos de Modificação: Push e Pop

Imagine a prateleira do mercado como um trem de vagões. O `push` **engatilha um novo vagão no final**, e o `pop` **desacopla o último vagão**. Ambos modificam o array original diretamente.

➕ push() — Adicionar ao final

```
produtos.push({
  id: 4,
  nome: "Água",
  preco: 2,
  estoque: 100
});
// produtos agora tem 4 itens
```

➖ pop() — Remover do final

```
let removido = produtos.pop();
// removido = { id: 4, nome: "Água"... }
// produtos volta a ter 3 itens
```

Atenção: Ambos os métodos **alteram o array original**. O `pop()` também **retorna** o item removido, o que pode ser muito útil!

BLOCO 2

O Cirurgião: splice()

Se `push` e `pop` trabalham apenas nas extremidades, o `splice` é o **cirurgião da lista** — ele opera em qualquer posição com precisão milimétrica. Podemos remover o "Chocolate" (que está no índice 1) assim:

splice() na Prática

Sintaxe do splice

```
array.splice(inicio, quantidade);
```

Removendo o Chocolate

```
let retirado = produtos.splice(1, 1);

// retirado = [{ id: 2, nome: "Chocolate"... }]
// produtos agora contém:
// [{ Coca-cola... }, { Sabonete... }]
```

Como ler os parâmetros:

- **1º parâmetro** → índice onde começa a operação (índice 1 = "Chocolate")
- **2º parâmetro** → quantos itens remover a partir dali (1 item)
- Também é possível **inserir novos itens** como 3º parâmetro e seguintes

⚠️ O `splice` **altera o array original**. Use com cautela!

BLOCO 3

Transformação com map()

O `map()` é como uma **linha de montagem**: cada item entra, é transformado por uma função, e sai como algo novo. O resultado é um **novo array** com o mesmo número de itens, mas com a "forma" que você definiu.

map() — Extraindo os Nomes dos Produtos

Código

```
let nomes = produtos.map(produto => produto.nome);

console.log(nomes);
// ["Coca-cola", "Chocolate", "Sabonete"]
```

Outro exemplo — aplicar desconto de 10%

```
let comDesconto = produtos.map(produto => ({
  ...produto,
  preco: produto.preco_venda * 0.9
}));
// Cada produto agora com preço reduzido
```

Como funciona

A função `produto => produto.nome` é chamada **para cada elemento** do array e representa o produto atual na iteração.

- Cria um **novo array** sem alterar o original
- O novo array sempre tem o **mesmo número de itens**
- Cada item é o **resultado da função** aplicada ao original

BLOCO 4

Seleção Estratégica com filter()

Pense em um filtro de café ☕

O `filter()` funciona exatamente como um filtro: o líquido (dados que passam no teste) escoa para o copo, e a borra (dados que não atendem ao critério) fica para trás.

O resultado é um **novo array** contendo **apenas os itens que passaram no teste** da função.

filter() — Produtos com Estoque Baixo

Queremos identificar quais produtos têm menos de 15 unidades no estoque para fazer um pedido de reposição.

Código

```
let estoqueBaixo = produtos.filter(
  produto => produto.estoque < 15
);

console.log(estoqueBaixo);
// [{ id: 3, nome: "Sabonete", preco: 3, estoque: 10 }]
```

Mais um exemplo — filtrar por preço

```
let baratos = produtos.filter(
produto => produto.preco <= 5);
// Retorna: Chocolate (R$5) e Sabonete (R$3)
```

Regras do filter()

- A função deve retornar **true** (item entra) ou **false** (item sai)
- Cria um **novo array** — o original não é alterado
- O novo array pode ter **menos itens** (ou nenhum) que o original
- Nunca terá **mais itens** que o array original

BLOCO 5

Resumo com reduce()

O `reduce()` é o **contador do caixa**: ele percorre todos os produtos, vai acumulando um valor e, no final, entrega o **resultado total**. É o mais poderoso — e também o que exige mais atenção para entender.

reduce() — Calculando o Valor Total do Estoque

Código completo

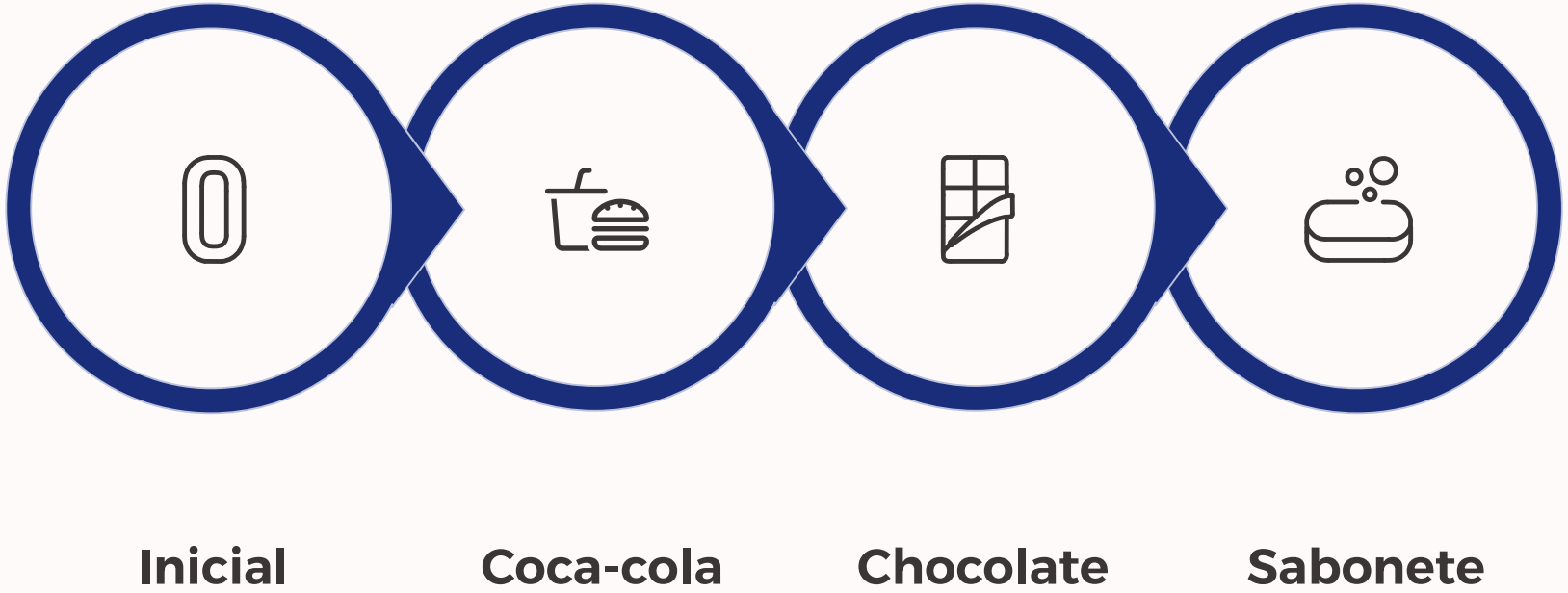
```
let totalEstoque = produtos.reduce(
  (acumulador, produto) => acumulador + (produto.preco_compra *
produto.estoque),
  0
);

console.log(totalEstoque);
// Coca-cola: 10 * 50 = 500
// Chocolate:  5 * 20 = 100
// Sabonete:   3 * 10 =  30
// TOTAL =          630
```

Entendendo os parâmetros

- **acumulador** → o "cofre" que guarda o resultado parcial a cada passo
- **produto** → o produto atual sendo processado
- **0** → o valor inicial do acumulador (começa do zero)

A cada iteração, o **acumulador** recebe o valor anterior somado ao novo cálculo. Ao final, o reduce retorna o acumulador com o resultado final.

Visualizando o reduce() Passo a Passo



O `reduce()` não pula etapas — ele processa **cada item do array em sequência**, carregando o acumulador adiante até o último elemento. O valor retornado é o acumulador final.

Comparando os 5 Métodos

push / pop

Adicionam ou removem itens **no final** do array. Alteram o original diretamente.

splice

Remove ou insere em **qualquer posição**. Alter o array original com precisão cirúrgica.

map

Transforma cada item. Retorna novo array com o mesmo número de elementos.

filter

Seleciona itens por critério. Retorna novo array com zero ou mais itens.

reduce

Resume o array inteiro em um único valor. Retorna qualquer tipo de dado.

Tabela Comparativa: Original vs. Novo Array

Uma das dúvidas mais comuns entre iniciantes é: **"Esse método altera meu array original ou cria um novo?"** Essa tabela responde de vez:

Método	Altera Original?	Cria Novo Array?	Retorna
`push()`	✅ Sim	❌ Não	Novo tamanho (number)
`pop()`	✅ Sim	❌ Não	Item removido
`splice()`	✅ Sim	❌ Não	Array com itens removidos
`map()`	❌ Não	✅ Sim	Novo array transformado
`filter()`	❌ Não	✅ Sim	Novo array filtrado
`reduce()`	❌ Não	❌ Não	Valor acumulado (qualquer tipo)

Dica de ouro: Métodos que **alteram o original** pedem cuidado redobrado. Já os que **criam novos arrays** são mais seguros para usar sem medo de perder dados!

Encadeando Métodos — O Poder Real

A mágica acontece quando você **combina métodos**! É possível encadeá-los um após o outro, criando pipelines de processamento elegantes.

Exemplo: nomes dos produtos baratos

```
let nomeBaratos = produtos
  .filter(p => p.preco <= 5)
  .map(p => p.nome);

// ["Chocolate", "Sabonete"]
```

Exemplo: total dos produtos em falta

```
let totalFalta = produtos
  .filter(p => p.estoque < 15)
  .reduce((acc, p) => acc + p.preco, 0);

// 3  (só o Sabonete passou no filtro)
```

Como ler o encadeamento

Leia da esquerda para a direita, de cima para baixo:

1. Começa com o array `produtos`
2. O resultado do `filter` alimenta o `map` (ou `reduce`)
3. O resultado final é atribuído à variável

Cada método recebe o **resultado do anterior** como entrada. É como uma linha de produção!

Conclusão: Você Agora Tem as Ferramentas!

🚂 push / pop

Vagões do trem — adicione e remova do final com facilidade.

🔪 splice

Cirurgião preciso — opere em qualquer posição da lista.

🏭 map

Linha de montagem — transforme cada item e gere uma nova lista.

☕ filter

Filtro de café — deixe passar só o que interessa.

🧮 reduce

Caixa registradora — some, acumule e resuma o array inteiro.

Próximos Passos: Pratique Agora!

1

Desafio 1 — push

Adicione um novo produto ao array `produtos`: `{ id: 4, nome: "Suco", preco: 4, estoque: 30 }`. Confirme com `console.log`.

2

Desafio 2 — filter

Filtre apenas os produtos com preço maior que R$ 4. Quantos produtos passaram pelo filtro?

3

Desafio 3 — map

Crie um novo array contendo apenas o campo `nome` e `preco` de cada produto. Use o `map()`.

4

Desafio 4 — reduce

Use o `reduce()` para contar a quantidade total de itens em estoque somando o campo `estoque` de todos os produtos.

💡 **Dica:** Você também pode resolver cada desafio no console do navegador (F12 → Console).

Resumo Final

Com esses cinco métodos você já é capaz de resolver a maioria dos problemas envolvendo listas de dados em JavaScript. Domine-os, pratique bastante e em breve você vai usá-los sem nem pensar — como um profissional! 🚀